readr cheat sheet

MC 451 / MC 501 / MC 427 · read a file in, write a file out

This sheet covers the commands that move data between a file on disk and a table in R. Each entry says what the command does in one line, shows it running on the course Twitch data, and shows the exact output R prints. The last two lines of each entry, **On your data**, rewrite the call with plain column names you can copy, and **If it breaks**, name the error you are most likely to see and what it means.

Every worksheet starts with these three lines, so the examples below assume them:

```
library(tidyverse)   # readr comes with the tidyverse
library(v2v)          # the course package and its Twitch data
options(scipen = 10)  # long numbers print in full
```

`read_csv()`, `write_csv()`, `read_rds()`, and `write_rds()` are all from **readr**. `clean_names()` is from **janitor**, which the wrangling worksheet also loads.

The data these examples use

A five-row slice of the chat messages, kept to three columns so it fits on the page:

```
coding_form <- chat %>%
  select(id, channel, message) %>%
  slice_head(n = 5)
coding_form
```

```
# A tibble: 5 × 3
     id channel    message
  <int> <chr>      <chr>
1    89 sodapoppin ??????????????????????
2   551 xqcow      TriEasy Clap TriEasy Clap TriEasy Clap TriEasy Clap TriE…
3  1033 forsen     cmonBruh
4  2094 sodapoppin sodaTHINKING
5  2803 xqcow      @xQcOW PLAY JACKBOX PARTY GAMES PLAY JACKBOX PARTY GAMES…
```

`write_csv()`: save a table as a plain `.csv`

Writes a table to a comma-separated file anyone can open in Excel or a text editor.

```
write_csv(coding_form, "data/coding-form.csv")
list.files("data")
```

```
[1] "coding-form.csv"
```

On your data: `write_csv(your_data, "data/your-file.csv")`

If it breaks: `cannot open file 'data/coding-form.csv': No such file or directory` means the `data` folder does not exist yet. Make it once with `dir.create("data")`, then write again.

`read_csv()`: read a `.csv` into a table

Reads a comma-separated file and guesses each column's type. It prints a short note about the types it guessed; that note is not an error.

```
codes <- read_csv("data/coding-form.csv")
codes
```

```
Rows: 5 Columns: 3
── Column specification ────────────────────────────────────────────────────────
Delimiter: ","
chr (2): channel, message
dbl (1): id
ℹ Use `spec()` to retrieve the full column specification for this data.
ℹ Specify the column types or set `show_col_types = FALSE` to quiet this message.
# A tibble: 5 × 3
     id channel    message
  <dbl> <chr>      <chr>
1    89 sodapoppin ??????????????????????
2   551 xqcow      TriEasy Clap TriEasy Clap TriEasy Clap TriEasy Clap TriE…
3  1033 forsen     cmonBruh
4  2094 sodapoppin sodaTHINKING
5  2803 xqcow      @xQcOW PLAY JACKBOX PARTY GAMES PLAY JACKBOX PARTY GAMES…
```

On your data: `your_data <- read_csv("data/your-file.csv")`

If it breaks: `does not exist in current working directory` means R is looking in a different folder than the one holding the file. Save the worksheet in your workspace's top folder so `data/` sits beside it.

Set the column types yourself

When read_csv guesses wrong, name the types yourself to silence the note. The string has one letter per column: `i` whole number, `d` number with decimals, `c` text, `l` true or false.

```
read_csv("data/coding-form.csv", col_types = "icc")
```

```
# A tibble: 5 × 3
     id channel    message
  <int> <chr>      <chr>
1    89 sodapoppin ??????????????????????
2   551 xqcow      TriEasy Clap TriEasy Clap TriEasy Clap TriEasy Clap TriE…
3  1033 forsen     cmonBruh
4  2094 sodapoppin sodaTHINKING
5  2803 xqcow      @xQcOW PLAY JACKBOX PARTY GAMES PLAY JACKBOX PARTY GAMES…
```

On your data: `read_csv("data/your-file.csv", col_types = "icc")`

If it breaks: the id column comes back as text with leading zeros kept (`007`) when you force `c`. Use `i` or `d` for anything you will do math on.

`clean_names()`: make column names safe to type

From **janitor**. Turns names with spaces, capitals, or punctuation into lowercase names that use _ between words, so you can refer to them without backticks.

```
messy <- tibble(`Message ID` = 1:3, `Reaction Only?` = c("yes","no","yes"))
messy %>% clean_names()
```

```
# A tibble: 3 × 2
  message_id reaction_only
       <int> <chr>
1          1 yes
2          2 no
3          3 yes
```

On your data: `your_data %>% clean_names()`, right after you read a file whose headers you did not write.

If it breaks: `could not find function "clean_names"` means janitor is not loaded. Add `library(janitor)` to the top of the file.

`write_rds()`: save a table in R's own format

Saves a table to an `.RDS` file that keeps every column type exactly, including dates and factors. Use it to hand a finished table to the next worksheet.

```
write_rds(analysis, "data/twitch_analysis.RDS")
file.exists("data/twitch_analysis.RDS")
```

```
[1] TRUE
```

On your data: `write_rds(your_data, "data/your-file.RDS")`

If it breaks: nothing visible happens on success, so check with `file.exists()` as above. A missing `data/` folder gives the same "No such file or directory" as `write_csv()`.

`read_rds()`: read an `.RDS` back in

Reads a table saved with `write_rds()` back into R, with every column type intact.

```
read_rds("data/twitch_analysis.RDS") %>%
  nrow()
```

```
[1] 35267
```

On your data: `your_data <- read_rds("data/your-file.RDS")`

If it breaks: `cannot open the connection` means the file is not where you pointed. The next worksheet reads `data/twitch_analysis.RDS`, so the name and folder must match exactly, capital letters included.

`problems()`: see which rows would not parse

After a read where a value did not fit its column type, `problems()` lists each one: the row, the column, what was expected, and what was found.

```
write_csv(tibble(count = c("12", "thirteen", "14")), "data/counts.csv")
counts <- read_csv("data/counts.csv", col_types = "i")
problems(counts)
```

```
Warning: One or more parsing issues, call `problems()` on your data frame for
details, e.g.:
  dat <- vroom(...)
  problems(dat)
# A tibble: 1 × 5
    row   col expected   actual   file
  <int> <int> <chr>      <chr>    <chr>
1     3     1 an integer thirteen data/counts.csv
```

On your data: run `problems(your_data)` whenever a read prints the parsing-issues warning.

If it breaks: `problems()` returns an empty table (`0 × 5`) when the read was clean. An empty result is good news, not a failure.

Also in the catalog

`read_excel()` (from **readxl**) reads a `.xlsx` spreadsheet the same way `read_csv()` reads a `.csv`; the R Companion writes it for you when your data arrives as an Excel file.